```
except UserProfile.DoesNotExist:
    # No UserProfile exists for this user, create one
    UserProfile.objects.create(user=user)
```

If no object existed, create one.

With your signal defined, you can test it out by signing in to the Django Admin and creating a new user named *vinny*. Within the Django Admin, you won't be able to tell the difference, as the `StackedInline` form is there, whether or not the `UserProfile` object exists. Inside a terminal, run the `shell` management command, and do the following:

```
>>> from django.contrib.auth.models import User
>>> User.objects.all()
<QuerySet [<User: admin>, <User: nicky>, <User: vinny>]>
>>> v = User.objects.last()
>>> v.userprofile
<UserProfile: UserProfile object (2)>
```

If your signal code had not worked, the last line in the REPL session would have caused an exception. Speaking of exceptions, having the signal handler in place does not fix past problems. Your admin account does not have a `UserProfile` associated with it. There are two things you can do to solve this problem. The first is to use the REPL and create a `UserProfile` object there; alternatively, you can use the Django Admin. Edit the admin user account, and create an association with a Musician object. Save it, and then go back and remove the association. The process of adding a `Musician` causes the Django Admin to create the `UserProfile` object for you.

The trouble with signals

Remember that ORM objects are proxies to the database. Signals are part of the ORM mechanic and are not tied to the database itself. Direct operations on your database will not emit a signal. Opening the SQLite REPL and adding a `User` by hand will result in a missing `UserProfile`.

Likewise, there are some bulk data operations available in Django that allow you to directly call SQL. These operations bypass the ORM layer and, therefore, also do not emit signals. Signals can be very handy, but you need to be cautious with the corner cases.

6.6 Password management

In addition to providing a view handling logging in, Django also provides views for dealing with password resets. This process emails the user an account reset link with a one-time token, and when the user clicks it, they are sent to a form to create a new password. Figure 6.6 shows the steps in the process.